Kenya Real Estate System Architecture

Technical blueprint for a custom listings and property management platform

21 October 2026
Architecture baseline version 1.0

Recommended foundation

Next.js and TypeScript for the web interface. NestJS for the application API. PostgreSQL and PostGIS for authoritative data. Object storage for media and documents. Durable workers for billing, payments and integrations.

Engineering focus

Organization isolation, tenancy constraints, financial correctness, M-PESA reconciliation and tested recovery. Includes a stack decision, data model, deployment plan, production tests and implementation roadmap.

Prepared for the developer building an independent Kenyan real estate product.

Contents

1 Purpose and recommended direction . . . 3

2 Recommended technology stack . . . 4

3 Application architecture and module ownership . . . 5

4 Network and deployment design . . . 6

5 Identity authorization and organization isolation . . . 8

6 Domain model and database schema . . . 9

7 Financial model and billing rules . . . 10

8 M PESA integration and reconciliation . . . 11

9 Reliable events and background work . . . 12

10 Search media and mobile experience . . . 13

11 API contracts and workflow states . . . 14

12 Kenyan privacy tax and operational compliance . . . 15

13 Security engineering and auditability . . . 15

14 Reliability backups and disaster recovery . . . 16

15 Monitoring support and daily operations . . . 17

16 Delivery testing and data migration . . . 17

17 Capacity cost and scaling plan . . . 18

18 Implementation roadmap and acceptance gates . . . 19

19 Research references . . . 21

1 Purpose and recommended direction

Build a custom Kenyan real estate platform with a modular NestJS backend, a Next.js web interface, PostgreSQL with PostGIS, object storage and separate background workers. Keep financial and tenancy decisions transactional. Use asynchronous processing for notifications, media, reconciliation and reporting. The primary engineering priorities are organization isolation, accurate financial records, reliable payment handling and demonstrated disaster recovery.

This blueprint covers public property listings and the ongoing management of buildings, units, leases, rent, maintenance and owner reporting. It consolidates the functional, network, database, operational and Kenyan integration requirements into one implementation plan. It is intended for the developer building and operating the product.

The baseline assumes a small development team building a platform that may serve multiple independent agencies or landlords. Traffic, managed-unit count, budget and existing language expertise must be established before final capacity commitments. Numerical sizing examples and service targets below are proposed planning assumptions, not measured performance or vendor quotations.

The implementation is independent of Odoo. Its earlier evaluation is retained only as a functional benchmark: connected property records, leads, contracts, recurring billing and maintenance are useful capabilities to reproduce. Odoo code, modules and hosting are not part of the proposed architecture. [25]

Product boundaries

- Public marketplace: published listings, photos, filters, map search, inquiries, saved properties and viewing requests.
- Agency workspace: portfolio, staff assignments, leads, listing moderation, reports and customer support.
- Landlord portal: authorized portfolio access, statements, expenses, maintenance approvals and settlement information.
- Renter portal: own lease information, invoices, receipts, payment status and maintenance requests.
- Platform administration: organization onboarding, abuse handling, operational diagnostics and audited support access.

Launch with residential long-term tenancy and ordinary sale/rental listings unless another segment is commercially necessary. Commercial leases, short-stay booking, installment land sales, escrow and pooled custody of funds introduce different accounting and legal workflows. Treat them as explicit scope extensions.

Decisions to settle before implementation

Decision	Recommended baseline	Why it matters
Customer structure	Multiple organizations with isolated records	Determines authorization and database keys
Collection account	Direct landlord or manager collection through approved provider arrangements	Determines reconciliation and settlement scope
Primary channel	Responsive web application	Supports customers and staff without separate native apps
Accounting scope	Operational receivables and a validated ledger	Avoids incorrect balances and owner statements

Decision	Recommended baseline	Why it matters
Availability	Defined service objectives with tested recovery	Determines infrastructure expense
Initial property segment	Residential long-term management plus listings	Keeps lease and billing rules bounded

2 Recommended technology stack

The default is TypeScript across the web and application layers. It reduces language switching and supports shared API contracts. The choice is a recommendation, not a claim that other languages cannot support this system. If the team is substantially more experienced in Python, PHP or C#, retain the same architecture and select an equivalent mature backend framework.

Layer	Recommended choice	Reason and constraint
Web interface	Next.js with React and TypeScript	Server-render public listings; interactive authenticated portals
UI styling	Tailwind CSS and an accessible component library	Consistent forms and responsive layouts; verify accessibility
Backend	NestJS on an actively supported Node.js LTS release	Explicit modules, validation, authorization and dependency injection
API	REST with OpenAPI	Clear contracts, generated clients and straightforward integration
Data access	Kysely with pg and reviewed SQL migrations	Type-safe queries while retaining control over PostgreSQL features
Primary database	Managed PostgreSQL with PostGIS	Transactions, relational integrity and map queries
Connection management	Bounded application pools; PgBouncer when needed	Prevent connection exhaustion; test transaction-pooling behavior
Background processing	BullMQ with a dedicated persistent Redis deployment	Worker scheduling and retries; database outbox remains replayable
File storage	Private S3-compatible object storage and public derivatives via CDN	Separate files from transactional data
Search	PostgreSQL full-text/trigram search and PostGIS initially	Add a dedicated search engine only when requirements justify it
Authentication	Established session or OIDC libraries with server-side authorization	Avoid implementing cryptography or authentication protocols
Observability	OpenTelemetry plus managed logs, metrics and error tracking	Correlate API requests, jobs and provider calls
Tests	Jest, real PostgreSQL integration tests, Playwright and k6	Domain correctness, browser behavior and representative load

Layer	Recommended choice	Reason and constraint
Packaging	Docker and a pnpm workspace	Repeatable builds and shared contracts
Delivery	GitHub Actions or equivalent CI and infrastructure as code	Reviewed migrations, reproducible deployment and auditability

Next.js supports server and client rendering and deployment as a Node.js application. NestJS provides the module, security and queue integration building blocks used here. Kysely is a TypeScript SQL query builder; the recommendation favors explicit SQL control for RLS, exclusion constraints and PostGIS. [1][2][3][4]

Pin exact dependency versions and container digests in the repository. Use supported stable releases, test upgrades in staging and maintain a patching cadence. Node.js recommends production use of supported LTS releases. Verify the supported PostgreSQL and PostGIS combination with the chosen host. [5]

Framework and database tradeoffs

Prisma or another ORM is acceptable if the team prefers it, but prove that migrations preserve custom SQL, RLS policies, range constraints and spatial indexes. Do not weaken the schema to fit an ORM. MongoDB is not the baseline because this domain depends heavily on relationships, constraints and multi-record financial transactions. A separate document database is unnecessary for ordinary leases and property attributes.

Django, Laravel and ASP.NET Core are viable alternatives based on team competence. Changing the backend language does not remove the need for idempotency, correct transactions and controlled authorization. Avoid running two backend frameworks for the same domain during the initial build.

Queue choice and durability

BullMQ is the default for a portable TypeScript deployment. Use persistent Redis, a no-eviction policy and monitored failover. Keep its deployment separate from an expendable application cache. Redis persistence and failover can still lose recent queue state, so retain replayable outbox records and reconcile unfinished business jobs. [6][7]

If deploying entirely on AWS, SQS is a reasonable managed-queue alternative. Choose one queue transport initially and put it behind a small adapter. Do not operate both without a concrete need. All workers must remain idempotent regardless of transport.

3 Application architecture and module ownership

Use a modular monolith with separately deployed web, API and worker processes. Modules share a PostgreSQL cluster but own their business operations. A transaction can span the closely related invoice, payment and ledger operations without a distributed transaction.

Module	Authoritative records	Main operations
Identity	Users, sessions, organizations, memberships	Invite, authenticate, revoke, assign permissions
Portfolio	Properties, units, ownership, management mandates	Register inventory, change management, record ownership
Listings	Listings, revisions, moderation decisions	Publish, withdraw, expire, revise
Leads	Inquiries, appointments, assignments	Assign lead, schedule viewing, record outcome

Module	Authoritative records	Main operations
Tenancies	Applications, leases, lease parties, inspections	Approve application, activate lease, amend, terminate
Billing	Charge schedules, occurrences, invoices, credits	Generate charge, issue invoice, adjust
Payments	Intents, attempts, provider transactions, allocations	Initiate, ingest, verify, reconcile, allocate
Ledger	Journals, accounts, deposit movements, settlements	Post, reverse, close period, prepare statement
Maintenance	Tickets, work orders, quotes, approvals	Assign contractor, approve expense, close job
Documents	File metadata, access grants, versions	Authorize upload, approve, download, retain
Integrations	Provider configuration and external references	Call adapters, track outcomes, retry safely

Controllers validate requests and call application services. Application services enforce authorization and transaction boundaries. Domain services enforce business rules. Repositories perform data access. Provider adapters translate external APIs without spreading provider-specific fields across the domain.

Use explicit commands such as activateLease, issueInvoice and postPayment. Avoid generic status updates that bypass transition checks. Reporting code can read approved projections but should not mutate source records. Do not let individual modules silently update each other's tables outside documented application operations.

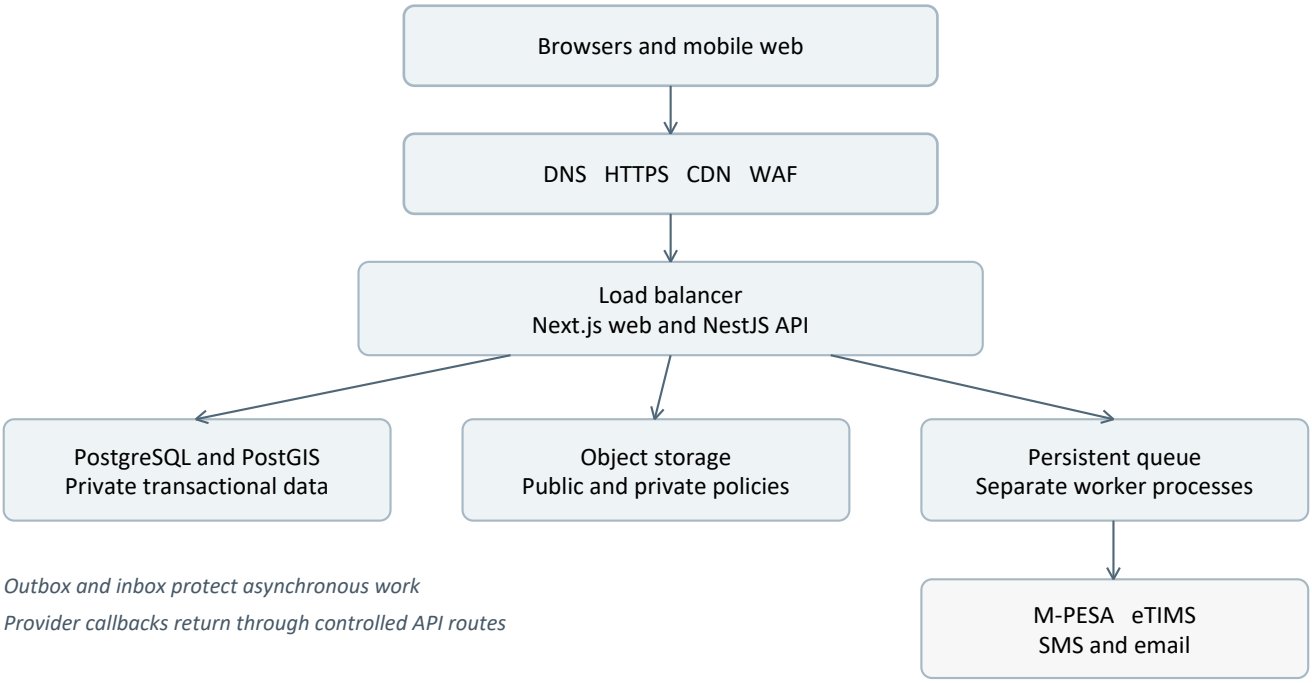
Suggested repository structure:

```
apps/web             Next.js public site and authenticated portals
apps/api             NestJS HTTP API and webhook endpoints
apps/worker          Queue consumers, dispatcher and schedulers
packages/contracts   API schemas, generated types and clients
packages/domain      Pure business rules and value objects
packages/db          Queries, SQL migrations and test fixtures
packages/integrations Provider adapters and contract tests
packages/config      Shared lint, build and test configuration
infra                Deployment definitions and runbooks
```

Keep database credentials and server-only modules out of web bundles. Next.js server code may compose pages, but NestJS remains the authoritative business layer. Avoid implementing different payment or permission rules in both systems.

4 Network and deployment design

The public network entry point terminates HTTPS, applies traffic controls and routes requests to the application. PostgreSQL, Redis, workers and administration endpoints remain private. Object storage grants access through narrowly scoped service identities or short-lived signed URLs. [8]



Outbox and inbox protect asynchronous work

Provider callbacks return through controlled API routes

Logical deployment and trust boundaries

Traffic and access matrix

Source	Destination	Rule
Internet	Edge	HTTPS 443; HTTP only for deliberate redirect
Edge or load balancer	Web and API	Application port restricted to trusted entry paths
API and workers	PostgreSQL	Private route and authorized role; TLS
Workers and API	Redis	Private route, authentication and TLS where supported
API and workers	Object store	Scoped service identity; block unintended public access
Provider	Webhook route	HTTPS plus provider-supported verification controls
Operations staff	Management plane	MFA and controlled administrative access
Internet	Database and Redis	Denied

Set body-size limits, request deadlines and concurrent-request limits at the edge and application. Keep callback routes free of browser login redirects or CAPTCHA. Validate request bodies and retain appropriate abuse controls on those routes.

Restrict direct origin access so the WAF cannot be bypassed. Trust forwarded headers only from known proxies. Configure CORS narrowly; CORS does not authorize access. Automate certificate renewal and domain renewal reminders. Store API credentials in a secrets manager, and prefer short-lived workload identities for infrastructure access.

Region and availability

Keep the API, workers and database in the same region. Measure latency from Kenyan networks before selecting a region. Evaluate privacy transfer requirements and support arrangements alongside latency. A CDN accelerates public content but cannot eliminate database round trips for private operations.

A cost-conscious pilot can use a managed container platform, managed PostgreSQL, persistent Redis and object storage. A single application instance is an acknowledged availability risk. For a stronger commercial baseline, use two application instances in separate failure domains, database failover, graceful deployments and health-based routing.

An AWS implementation can map these components to ECS Fargate, RDS PostgreSQL, S3, CloudFront, WAF, a load balancer and a managed queue or persistent Redis. This is a reference mapping, not a requirement to use AWS. Managed hosting reduces infrastructure work but still requires secure configuration and recovery planning. [9]

Budget for NAT or other outbound routing charges, cross-zone traffic, backup storage and logs. If providers require outbound IP allowlisting, design fixed egress deliberately. Office internet should not host the public service; staff connectivity can use backup internet, a UPS and separate guest Wi-Fi.

5 Identity authorization and organization isolation

Use organization for a SaaS customer and renter or lease_party for a person occupying a property. One user can belong to several organizations, while each organization-owned record carries its ownership scope.

Start with shared tables and organization_id, application authorization and PostgreSQL row-level security. Derive the organization context from the authenticated user's current membership. Never accept an arbitrary request header or body field as sufficient authority. [10][11]

Use composite references such as (organization_id, unit_id) referencing (organization_id, id) to prevent cross-organization relationships. Scope private cache keys, storage downloads, queue work and reports. Public listing projections should deliberately contain only publishable fields.

RLS must run under an application role that is not a superuser, table owner or BYPASSRLS role. Evaluate FORCE ROW LEVEL SECURITY where appropriate. Put organization context inside the database transaction and clear it safely through transaction boundaries. With PgBouncer transaction pooling, do not assume a connection retains session configuration. Test policies using the real runtime role. [11][12]

Actor	Typical access	Important restriction
Agency manager	Own organization's portfolio and staff	No access to other organizations
Agent	Assigned leads and permitted listings	Financial access requires separate permission
Accountant	Billing, reconciliation and statements	Sensitive corrections require approval
Landlord	Assigned ownership or management portfolio	No other owner's private records
Renter	Own lease, invoices and requests	No neighboring renter information
Contractor	Assigned work orders	Minimum necessary contacts and attachments
Platform support	Approved time-limited support session	Every access is audited

Use role-based permissions combined with relationship checks. The role landlord alone does not identify which properties a user may access. Revocation should take effect for queued exports, download links and active sessions according to an explicit policy.

For browser access, prefer established server-side session handling with Secure, HttpOnly cookies, appropriate SameSite settings and CSRF protection. Use standard OAuth/OIDC flows if native apps or external clients are introduced. Require MFA for administrators and finance staff, protect recovery flows and require reauthentication for payout-destination changes. [13]

Add quotas for files, exports, SMS and queued work. Organization isolation includes preventing one customer's large import from exhausting shared resources. Database-per-organization isolation can be introduced for customers with stronger contractual requirements, but it increases migration, backup and operating complexity.

6 Domain model and database schema

Separate the physical asset, advertisement, contractual tenancy and financial activity. A property can have several units. A unit can have multiple historical listings and leases. Ownership and management change over time without deleting the past.

Area	Core tables
Identity	users, organizations, memberships, roles, permissions, sessions
Portfolio	properties, units, ownership_interests, management_mandates
Marketplace	listings, listing_revisions, media, amenities, listing_amenities
Leads	inquiries, lead_assignments, viewing_appointments, saved_listings
Tenancy	applications, leases, lease_parties, lease_amendments, inspections
Billing	charge_schedules, billing_occurrences, invoices, invoice_lines, credit_notes
Payments	payment_intents, payment_attempts, provider_transactions, payment_allocations
Accounting	ledger_accounts, journal_entries, journal_lines, deposit_movements, settlements
Operations	maintenance_requests, work_orders, approvals, document_versions
Reliability	webhook_inbox, outbox_events, processed_events, reconciliation_runs
Governance	audit_events, retention_rules, access_requests, import_batches

Essential field conventions

- Use UUIDs or another opaque identifier for externally exposed entities. IDs are not authorization controls.
- Store money as NUMERIC with a defined precision/scale, or integer minor units, and a currency code. Avoid JavaScript floating-point arithmetic for financial calculations; use decimal-safe arithmetic and explicit rounding rules.
- Use timestamptz for events and date for contractual and billing dates. Display business time in Africa/Nairobi. Record both effective_at and recorded_at when late entry matters.
- Keep core filter and accounting fields typed and constrained. Use JSONB only for supplementary attributes with documented validation.

- Record created_by, relevant change reasons, version and external references. Use version-based optimistic concurrency where staff may edit the same record.
- Restrict deletion of financial and contractual records. Soft deletion is not a privacy-retention policy; implement separate retention and erasure processes.

An illustrative unit record contains id, organization_id, property_id, unit_code, unit_type, bedrooms, bathrooms, floor_area, area_unit and operational_status. A listing contains its own price, publication state and content revision. A lease stores agreed terms independently of the current listing price.

Ownership interests and management mandates need effective dates. Preserve historical access and reporting rules during a management transfer. An agent losing authority must not retain access simply because they originally created the record.

Store geographic coordinates separately from human-readable locality fields. Support county, subcounty/locality, neighborhood, address and location notes without assuming every property has a formal street address. Public map positions can be less precise than private operational locations.

Constraints and concurrency

Enforce unique unit codes within a property, scoped provider transaction references, billing-occurrence uniqueness and cross-organization foreign keys. Require positive allocations and valid date ranges. Cross-row rules such as allocation totals or balanced journals need a controlled posting transaction and suitable database enforcement; a simple row CHECK cannot enforce a sum across other records.

For exclusive occupancy, represent the occupied period with a date range and an exclusion constraint on organization, unit and overlapping range. Include only states that actually reserve occupancy. Define whether contract end dates are inclusive and convert them consistently to half-open ranges. Shared accommodation must reserve beds/rooms or use explicit capacity rules. [14]

Do not use a separate availability check followed by an unprotected insert. Concurrent requests can both pass. Use constraints and appropriate locks or serializable transactions with bounded retries. Keep transactions short and avoid network calls while holding locks. [15]

7 Financial model and billing rules

Build an operational receivables subsystem with a validated ledger. An editable renter.balance field is not sufficient as the source of truth. Balances should be derived from issued charges, credits, allocations and financial postings, with cached summaries that can be rebuilt.

A payment and an invoice are different records. One payment can cover several invoices; one invoice can receive several payments. Keep allocations explicit. Unapplied money remains identifiable as credit or unmatched funds rather than disappearing into a balance.

Example event	Amount in KES	Result
Monthly rent invoice	25,000	Receivable created
Confirmed partial payment	10,000	Payment recorded
Allocation to rent invoice	10,000	Outstanding rent becomes 15,000
Security deposit received	25,000	Separate deposit liability or appropriate account
Later correction	Defined adjustment	Reversal or credit preserves original history

Account treatment depends on whether the organization owns the property or collects as an agent. A Kenyan accountant should approve the chart of accounts, owner liabilities, revenue recognition, taxes and settlement rules. Do not treat rent collected for someone else as the platform's revenue.

Billing decisions to specify

- Monthly billing dates, leap years and month-end behavior.
- Proration for partial occupancy and the exact day-count convention.
- Grace periods and approved late-fee rules.
- Rent increases with effective dates and signed amendments.
- Utility readings, estimates, meter replacement and correction.
- Allocation order across rent, utilities, service charges and penalties.
- Deposits, deductions, refunds and disputed deductions.
- Management fees, approved expenses and landlord payout timing.
- Credit notes, write-offs, closed periods and backdated transactions.

Create billing_occurrences with a unique business key such as charge_schedule_id plus period boundaries. A scheduler can safely reprocess a period after a restart. Issue each invoice once and record the ledger posting and outbox notification in the same transaction.

Posted journals should be immutable through ordinary application paths. Corrections use linked reversals and replacement entries. Require balanced debits and credits per journal and currency before posting. Use an approved workflow for refunds, write-offs and payout-destination changes, with separation of duties where staffing allows it.

Keep invoice numbering and fiscal numbering distinct if the integration requires separate sequences. Do not infer tax compliance from the existence of a PDF invoice. Define an accountant-reviewed period-close procedure and reconciliation report before onboarding real rent payments.

8 M PESA integration and reconciliation

Integrate through a provider adapter with separate production and sandbox credentials. Model payment_intent, payment_attempt, provider_transaction, payment_allocation and settlement independently. This preserves the difference between a user requesting payment, a provider processing it and the system applying money to debt.

M-PESA Express initiates a customer payment request. C2B also receives payments initiated outside your website. C2B confirmation and optional validation have different purposes; optional validation must be enabled through the provider's supported process. [16][17]

```
Create intent and attempt
  -> Submit provider request and persist correlation identifiers
  -> Receive callback into durable webhook inbox
  -> Validate evidence and identify organization and merchant
  -> Deduplicate provider transaction
  -> Post payment and allocations in one database transaction
  -> Add receipt event to outbox
  -> Reconcile against provider records
```

Callback handling

Persist accepted callback input durably before acknowledging it. Validate body size and schema, keep sensitive payloads access-controlled and use the chosen provider's actual authentication controls. Do not invent an HMAC verification scheme if the endpoint does not provide signatures. HTTPS protects transport but does not by itself establish the sender's identity.

Correlate merchant/account, request identifiers, amount, currency and receipt reference. The phone number alone must not identify the invoice or prove payment. Treat suspicious or unmatched events as exceptions requiring verification. Use the relevant provider query or reconciliation mechanism for the exact payment flow. Safaricom's Transaction Status API supports applicable transaction inquiries using supported identifiers. [18]

Use a unique key scoped to provider, merchant account and provider transaction reference as appropriate to the provider contract. Two callback formats may describe the same receipt, so deduplication by callback ID alone is insufficient. Late duplicates must remain harmless after queue history is cleaned up.

Failure states

Event	Required behavior
Request accepted	Show pending rather than paid
Request times out	Mark unknown and reconcile before retrying
Callback arrives twice	One payment and one allocation effect
Callback arrives before request response	Retain inbox event and match when identifiers are available
Success arrives after local timeout	Resolve pending outcome without creating another charge
User pays outside the app	Record incoming payment and match account reference
Account reference is unknown	Place in unmatched queue with audited manual allocation
Payment is reversed	Create linked reversal and update affected balances
Provider is unavailable	Preserve local state and retry only safe operations

Keep initiation state separate from settlement evidence. A single boolean paid field cannot represent asynchronous processing. Returning 200 to a callback does not guarantee its downstream work completed; the inbox and workers must provide that guarantee.

For outgoing refunds or owner payouts, require approval, destination verification and durable correlation IDs. After a timeout or restore, query/reconcile before retrying. A database restore cannot undo a real payment already executed by the provider.

Decide whether collection is direct to landlords/managers or handled through your platform. Custody and distribution of funds changes the provider relationship and may change regulatory obligations. Confirm that operating model before building wallet balances or pooled settlement. CBK publishes authorized payment service providers and approved activities. [19]

9 Reliable events and background work

Use the transactional outbox pattern when a database change must trigger external work. Write the domain change and event in one transaction. A dispatcher publishes committed events. Workers record processed event IDs and apply idempotent business operations. Delivery can be repeated even with a reliable broker. [20]

Retain event_id, organization_id, aggregate_id, event_type, schema_version, occurred_at and payload. Avoid copying unnecessary private information into queue messages. Event versions need backward-compatible consumers during deployments.

For external callbacks, use an inbox table containing provider reference, received_at, validation state, processing state and payload reference. Acknowledgment follows durable storage, while business processing may continue asynchronously. Failed events must be visible to authorized operations staff.

Queue class	Work	Policy
Financial	Callback processing, billing, reconciliation	Highest priority and bounded concurrency
Communications	SMS, email, receipts	Retries with provider-aware throttling
Media	Image validation and derivatives	Resource limits and separate workers
Reporting	Statements, exports, analytics refresh	Organization quotas and cancellable long jobs

Use exponential backoff with jitter, maximum attempts, processing deadlines and a dead-letter or equivalent failed-job workflow. Acknowledge only after durable work commits. Graceful worker shutdown should stop taking new jobs and allow current work to finish or be safely redelivered.

Periodic tasks should catch up from database state, not rely on one cron invocation firing at the perfect time. Track expected versus completed billing occurrences. A daily reconciliation can detect a queue job lost during failover even if the dispatcher previously marked it sent.

10 Search media and mobile experience

Start search with relational filters, PostgreSQL text search and PostGIS. Index the actual access patterns, for example published rental listings by locality, unit type and price. Use EXPLAIN with realistic data and parameter distributions. Avoid adding every conceivable composite index because indexes increase write and maintenance cost.

Spatial queries should use suitable GiST indexes and index-aware functions. ST_DWithin on geography provides metre-based radius semantics. Limit map results, cluster dense markers, debounce panning and use consistent pagination. A dedicated search engine becomes appropriate when measured needs for relevance, typo tolerance or faceting exceed the initial approach. [21]

Search indexes and caches are derived data. Lease activation and financial operations must query authoritative state. Publish listing changes through the outbox and periodically compare published status with index contents to remove stale advertisements.

Media storage

Store binaries in object storage and metadata in PostgreSQL. Separate public derivatives, private originals, contracts, identity documents and generated statements. Authorize every private download before issuing a short-lived signed link. URLs can remain valid until expiry, so expiration is part of access policy. [22]

The upload lifecycle is authorization, temporary upload, content validation, scanning/processing, approval and publication. Validate file signatures and dimensions, generate object names, cap sizes and remove unnecessary EXIF metadata. Serve user files from an appropriately isolated origin. Do not trust a filename extension or browser MIME type alone. [23]

Track object version, checksum, uploader, organization, classification and retention status. Reconcile orphaned uploads and failed processing. Backups must preserve the relationship between database metadata and object versions.

Mobile and discovery

Server-render public listing pages, use canonical URLs and control indexing of filter combinations. Keep private portals out of search indexes, while relying on authorization for actual security. Include accessibility checks for keyboard navigation, labels, contrast and error messages.

Use responsive image sizes and load secondary images lazily. Preserve safe form drafts on connection failure. Payment retries should reuse the existing intent and fetch its state after reconnecting. Avoid automatically resubmitting a financial operation when a browser comes back online.

Offline maintenance drafts are reasonable. Require server confirmation for lease activation, refunds and financial posting initially. Do not cache sensitive financial pages or identity documents in a service worker by default.

Target the documented Core Web Vitals thresholds at the 75th percentile: LCP within 2.5 seconds, INP at most 200 milliseconds and CLS at most 0.1. Measure separately on mobile and desktop. These are user-experience targets rather than API latency limits. [24]

11 API contracts and workflow states

Expose REST endpoints that correspond to meaningful business operations. Publish OpenAPI and generate client types, but retain server-side validation. Reject unknown or forbidden writable fields to prevent mass assignment.

```
POST /organizations/{id}/invitations
POST /listings/{id}/publish
POST /leases/{id}/activate
POST /invoices/{id}/issue
POST /payment-intents
POST /payments/{id}/allocations
POST /refund-requests/{id}/approve
GET  /exports/{id}/status
```

Use idempotency keys for retried commands, scoped to authenticated principal or organization and operation. Store a request hash and result reference. Reusing a key with a different payload should be rejected. Handle simultaneous requests for the same key atomically. Retention must cover the realistic retry window; durable financial reference uniqueness remains necessary afterward.

Return consistent error codes, safe messages and request IDs. Use 409 or another documented conflict response for concurrent version conflicts. Bound page sizes and expensive filters. Generate large exports asynchronously and recheck authorization before exposing them.

Record	Example states	Critical guard
Listing	Draft, review, published, withdrawn, expired	Only approved publishable content is public
Lease	Draft, approved, active, ended, cancelled	Activation enforces occupancy and signed terms
Invoice	Draft, issued, voided with controls	Issued values change through formal adjustments
Payment attempt	Created, pending, unknown, succeeded, failed	External evidence determines outcome
Maintenance	Open, triaged, assigned, in progress, completed, closed	Cost approval and completion evidence are distinct

Document allowed transitions, required permissions and audit events. A status string must not let an arbitrary request bypass business rules. Invoice payment status can be derived from allocations rather than maintained independently by multiple processes.

12 Kenyan privacy tax and operational compliance

Map controller and processor responsibilities for the platform and each customer. Review ODPC registration requirements; its guidance specifically addresses categories including property management. Record the purposes and lawful bases for processing personal data. Minimize identity-document collection and restrict sensitive exports. [26]

Implement data access and correction requests, retention schedules, deletion or anonymization where applicable, and legal holds. Erasure must respect lawful retention requirements and propagate to derived stores. Backup expiry and restoration procedures should prevent deleted data from silently returning to normal use.

Review cross-border safeguards for hosting, backups, email, error monitoring and other processors. Kenyan data protection rules do not reduce to a blanket yes/no choice about overseas servers. Assess the actual data flow and applicable safeguards. Maintain an incident response and breach-reporting process with assigned responsibility. [27]

eTIMS architecture

KRA provides OSCU and VSCU routes for system-to-system integration and describes testing and certification for self-integrators and third-party developers. Determine the appropriate route and invoice scope with an accountant and the current KRA onboarding process. Ordinary receipt generation is not proof of fiscal compliance. [28]

Maintain fiscal_submission records separately from internal invoice state: pending, submitted, accepted, rejected and requires intervention. Store provider identifiers, payload version, response and retry metadata. Respect the chosen protocol's sequencing and replay rules. Keep configurations isolated by taxpayer, branch or device as required by that integration.

Do not mark a failed fiscal submission accepted merely to let the workflow continue. Provide staff visibility and controlled correction. Business-specific tax rates, exemptions and rent/agency treatment should be configurable after professional validation rather than embedded as unreviewed assumptions.

Real estate trust and moderation

Include agent/landlord verification, documented authority to list, listing reports, duplicate review, stale-listing expiry and moderation history. Publishing a listing or displaying a badge must not falsely imply verified legal title. Keep verification criteria and evidence explicit.

Separate transactional notifications from marketing preferences. Record consent where required, support opt-out and limit SMS abuse. A staff member's ability to export customer contacts should be an explicit permission with an audit trail.

13 Security engineering and auditability

Threat-model cross-organization access, account takeover, fraudulent payment callbacks, malicious uploads, abusive inquiries, compromised integration keys and privileged misuse. OWASP's API guidance highlights object authorization and resource-consumption risks that directly apply here. [29]

- Enforce authorization at every read and write path, including exports, jobs and file downloads.
- Use parameterized SQL, output encoding, a suitable content security policy and maintained dependencies.
- Protect URL-import or image-fetch features against SSRF, or omit remote fetching initially.
- Keep secrets out of source control, browser bundles, logs and database exports.
- Rate-limit login, password recovery, inquiries, uploads, payment initiation and exports.
- Require approvals for refunds, write-offs and sensitive destination changes.
- Record actor, organization, action, target, timestamp, reason and correlation ID for sensitive events.

Keep audit events access-controlled and export them to storage with restricted alteration. An application audit table alone is not tamper-proof against a privileged database administrator. Ordinary users must not edit or delete audit history.

Use a separate runtime database role from migration administration. Review security-definer functions and any RLS bypass carefully. Test unauthorized object access by changing IDs, organization selection and file references. Test disabled accounts and revoked memberships, not just valid happy-path roles.

Define security patch ownership, incident triage, credential rotation and evidence-preservation procedures. Support access should use explicit permission and a recorded reason rather than a permanent unrestricted impersonation button.

14 Reliability backups and disaster recovery

Define service objectives before selecting availability features. The following are proposed launch targets to validate through load and recovery tests.

Measure	Proposed target	Measurement
Core application availability	99.9 percent monthly	Synthetic checks of named workflows
Ordinary API latency	p95 below 500 ms	Server latency excluding defined external waits
Search API latency	p95 below 1 second	Representative filters and dataset
Recovery point objective	At most 15 minutes	Maximum lost database changes in disaster scenario
Recovery time objective	Within 4 hours	Timed restoration and verification exercise
Duplicate payment posting	Zero	Uniqueness checks and reconciliation
Unauthorized organization access	Zero	Security tests and incident monitoring

At 99.9 percent, a 30-day month allows approximately 43 minutes of unavailability. Specify whether provider outages count against each user-facing objective. A provider's SLA alone does not establish your application's availability.

Enable automated backups and point-in-time recovery with an appropriate retention policy. PostgreSQL PITR combines base backups and archived WAL. A replica can reproduce accidental deletion and is not an independent backup. Validate that the actual managed plan includes the required recovery features. [30]

Back up object versions, infrastructure definitions and critical configuration. Preserve access to encryption keys through a separate recovery procedure. Keep backup credentials and deletion permissions distinct from routine application privileges where possible.

Recovery runbook

1. Declare the incident, stop unsafe writes and identify the recovery boundary.
2. Restore the database into an isolated environment and verify integrity.
3. Restore or reconnect the matching file objects and required secrets.
4. Reconcile payments and payouts executed externally after the recovery point.

5. Resume inbox/outbox processing and missed billing occurrences safely.
6. Rebuild derived search/cache data, validate permissions and run smoke tests.
7. Restore traffic gradually and monitor business as well as infrastructure signals.

A successful restore is measured by correct recoverable business operations. Perform it before launch and periodically afterward. Record the observed recovery time and data gap, and revise the promised objectives if the evidence does not support them.

15 Monitoring support and daily operations

Instrument API and worker processes with OpenTelemetry traces, metrics and logs. Propagate request IDs through jobs and provider calls. Do not place phone numbers, identity documents, secrets or full payment payloads into routine logs. [31]

Monitor latency, traffic, errors and saturation, following the established SRE monitoring model. Add business checks because a healthy process can still fail to generate rent invoices or reconcile payments. [32]

Signal	Operational response
Growing unresolved payment age	Review provider health and reconciliation backlog
Expected invoices exceed issued invoices	Inspect scheduler and billing failures
Oldest financial queue job increases	Scale workers or resolve blocked processing
Database pool saturated	Check slow queries and total connection budget
Reconciliation difference	Investigate unmatched, duplicate or reversed transactions
Backup or restore check failure	Escalate before recovery capability is assumed
Fiscal submissions rejected	Route to integration/finance exception workflow
Withdrawn listing remains public	Repair projection or cache invalidation

Set alerts on user-impacting symptoms and sustained thresholds, with a named responder and runbook. Avoid generating an alert for every recoverable retry. Keep retention and cost limits on verbose logs and traces.

Build an operations console for failed-job inspection, unmatched payment allocation, fiscal errors, listing moderation and support access. Every manual correction should call the same domain services and produce audit history. Direct production SQL should be an exceptional controlled recovery measure.

16 Delivery testing and data migration

Use isolated development, staging and production environments. Staging should exercise the same PostgreSQL features, queue behavior and permissions as production, using synthetic or appropriately anonymized data. Keep sandbox payment credentials and callback URLs distinct from production.

A pipeline should build immutable artifacts, run lint/type checks, domain tests, database integration tests, browser tests and security checks, then deploy through controlled promotion. Run migrations as a dedicated reviewed step rather than racing them from every application instance.

Use expand-and-contract migrations: add compatible structure, deploy compatible code, backfill in bounded batches, switch reads/writes and remove old fields later. Test migration lock behavior and rollback compatibility. Rolling back code does not undo a destructive data migration.

Required test scenarios

Scenario	Pass criterion
Concurrent overlapping lease activation	One valid occupancy reservation
Duplicate callbacks and receipt formats	Exactly one financial effect
Crash after domain commit	Outbox work eventually completes
Billing rerun or scheduler restart	No duplicate charge occurrence
Provider timeout after acceptance	No blind repeated collection or payout
Agency A requests B's resource	Denied across API, files and exports
Revoked member has a queued export	Delivery rechecks authorization
SMS provider outage	Financial save succeeds; message retries
Restore after external payment	Reconciliation prevents repeated effects
Closed-period correction	Authorized linked adjustment preserves history
Queue failover loses recent job	Database reconciliation can recover work
Large report during payment peak	Financial processing remains within targets

Use real PostgreSQL for RLS, range and concurrency tests. Add property-based tests for ledger balancing, allocation conservation and rounding. Browser tests should cover mobile layouts, keyboard operation, reconnect behavior and file flows. Playwright supports cross-browser end-to-end testing. [33]

For legacy spreadsheet imports, preserve the original file, validate into staging tables, show errors, support a dry run and assign an import batch ID. Deduplicate with explicit rules. Import opening balances as approved financial entries and reconcile totals with the source. Make re-imports idempotent and retain provenance.

17 Capacity cost and scaling plan

Size by workload rather than registered-user count. Record published listings, managed units, active leases, peak requests per second, callbacks per minute, billing volume, image volume, report size and retention. Use realistic mixed load with rent-day billing and public browsing occurring together.

The following examples are arithmetic assumptions, not capacity guarantees:

Workload assumption	Approximate result
50,000 listings with 12 images of 300 KB	180 GB of derivatives before originals and backups

Workload assumption	Approximate result
100,000 page views transferring 1.5 MB each	150 GB of transfer before cache and compression effects
10,000 recurring invoices processed over 1 hour	Average 2.8 invoices per second; peak and posting cost still matter
4 API replicas with 10 connections each plus 2 workers with 5	50 database connections before administration and other processes

Set an explicit connection budget. Autoscaling application replicas without limiting pools can exhaust PostgreSQL. Bound worker concurrency separately from web concurrency and reserve capacity for critical operations.

Cost worksheet

Estimate compute, managed database, standby/failover, backups, object storage, CDN transfer, queue/Redis, SMS, email, maps, monitoring, staging and provider transaction charges. Add developer maintenance and support time. Obtain current quotes for the selected region and commercial plans before committing to a monthly figure.

Track unit economics such as hosting cost per managed unit, messaging cost per collection cycle and bandwidth cost per listing view. Set billing alerts and usage quotas to detect abuse or accidental export loops.

Scaling sequence

First fix query plans, N+1 requests and oversized responses. Then move heavy work to workers, deliver public media through the CDN and cache safe read projections. Scale API and worker processes independently. Introduce reporting replicas, dedicated search or partitioning only when measured bottlenecks justify them.

Read replicas can lag. Keep financial confirmation, authorization-sensitive reads and immediate post-write views on an appropriate authoritative path. A cache outage should have a bounded fallback that does not overwhelm the database.

Extract microservices when a domain has an independent team, deployment cadence or scaling need. Kafka, Kubernetes, active-active multi-region writes and event sourcing are deferred choices rather than launch requirements.

18 Implementation roadmap and acceptance gates

Phase	Deliverable	Exit evidence
1 Foundation	Identity, organizations, migrations, CI, secrets and audit	Organization isolation tests and reproducible deployment
2 Inventory and marketplace	Properties, units, media, moderation, search and leads	Publish/withdraw flow and private-file access tests
3 Tenancy	Lease lifecycle, parties, inspections and amendments	Concurrent occupancy constraints and history validation
4 Financial core	Billing, allocations, ledger, deposits and statements	Accountant-reviewed examples and rerun-safe billing
5 Payments	M-PESA, inbox/outbox, reconciliation and exceptions	Duplicate, delayed and missing callback scenarios

Phase	Deliverable	Exit evidence
6 Operations	Maintenance, notifications, reports and eTIMS scope	Staff exception workflows and provider acceptance
7 Production readiness	Load, security, recovery and support runbooks	Timed restore, mixed-load results and alert verification

Build a thin end-to-end tenancy example early: create a unit, publish it, sign a lease, generate rent, accept a partial payment, allocate it and produce a statement. Expand the system after that path has reliable transactions and clear ownership.

Launch checklist

- Organization isolation passes for every private API, file, export and worker path.
- Financial rules and account mapping are reviewed using realistic Kenyan examples.
- Billing and payment processing remain correct under retries and crashes.
- Merchant onboarding, live credentials and callback routing are verified.
- Unknown and unmatched payments have an operational resolution workflow.
- Backups, files, secrets and external reconciliation are covered by a tested restore.
- Production monitoring has a responder and actionable runbooks.
- Privacy, tax and collection arrangements are confirmed for the business model.
- Security patching, access reviews and incident handling have owners.
- Load tests establish capacity limits and cost alerts are active.

Architecture decision register

Record short decisions in the repository for the stack, organization isolation, ledger scope, money flow, queue transport, hosting region, file retention and recovery objectives. Each decision should include its reason, alternatives, consequences and conditions for revisiting it.

The next implementation inputs are the team's strongest backend language, managed-unit and listing estimates, expected peak traffic, monthly infrastructure budget, preferred hosting region and direct-versus-platform collection model. These refine this baseline without changing its correctness requirements.

19 Research references

Sources were consulted on 21 October 2026. Product documentation and integration onboarding can change; confirm the exact deployed versions and current provider contracts during implementation. Architecture choices and proposed targets in this document are recommendations derived from the requirements, not vendor guarantees.

[1] Next.js application architecture and rendering.
https://nextjs.org/docs/app/getting-started

[2] Next.js deployment options.
https://nextjs.org/docs/app/getting-started/deploying

[3] NestJS official documentation.
https://docs.nestjs.com/

[4] Kysely SQL query builder.
https://kysely.dev/

[5] Node.js release and support policy.
https://nodejs.org/en/about/previous-releases

[6] NestJS queue integration.
https://docs.nestjs.com/techniques/queues

[7] BullMQ production guidance.
https://docs.bullmq.io/guide/going-to-production

[8] AWS web and database network architecture.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-web-database-servers.html

[9] Amazon RDS concepts and architecture.
https://docs.aws.amazon.com/AmazonRDS/latest/gettingstartedguide/concepts.html

[10] OWASP multi-tenant security.
https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html

[11] PostgreSQL row security policies.
https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[12] PgBouncer pooling behavior.
https://www.pgbouncer.org/features.html

[13] OWASP authentication guidance.
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[14] PostgreSQL range types and exclusion constraints.
https://www.postgresql.org/docs/current/rangetypes.html

[15] PostgreSQL transaction isolation.
https://www.postgresql.org/docs/current/transaction-iso.html

[16] Safaricom Customer to Business API.
https://developer.safaricom.co.ke/apis/CustomerToBusiness

[17] Safaricom M-PESA Express API.
https://developer.safaricom.co.ke/apis/MpesaExpressSimulate

[18] Safaricom Transaction Status API.
https://developer.safaricom.co.ke/apis/TransactionStatus

[19] Central Bank of Kenya payment service providers.
https://www.centralbank.go.ke/2022/09/14/directory-of-payment-service-providers/

[20] AWS transactional outbox pattern.
https://docs.aws.amazon.com/en_en/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[21] PostGIS distance queries.
https://postgis.net/docs/ST_DWithin.html

[22] S3 presigned access URLs.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

[23] OWASP file-upload controls.
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[24] Google Web Vitals.
https://web.dev/articles/vitals

[25] Odoo property management functional benchmark.
https://www.odoo.com/industries/property-management

[26] ODPC registration guidance.
https://www.odpc.go.ke/wp-content/uploads/2024/02/ODPC-Guidance-Note-on-Registration-of-Data-Controllers-and-Data-Processors.pdf

[27] ODPC data subject rights.
https://www.odpc.go.ke/rights-of-a-data-subject/

[28] KRA eTIMS system-to-system integration.
https://www.kra.go.ke/business/etims-electronic-tax-invoice-management-system/learn-about-etims/etims-system-to-system-integration

[29] OWASP API Security Top 10.
https://api-security.owasp.org/editions/2023/en/0x00-header/

[30] PostgreSQL continuous archiving and PITR.
https://www.postgresql.org/docs/current/continuous-archiving.html

[31] OpenTelemetry signals.
https://opentelemetry.io/docs/concepts/signals/

[32] Google SRE monitoring distributed systems.
https://sre.google/sre-book/monitoring-distributed-systems/

[33] Playwright browser testing.
https://playwright.dev/docs/intro